

สัญญา tRPC

คืออะไร และเราต้องทำอะไรบ้าง

หลักการของ tRPC · องค์ประกอบที่ต้องมีให้ครบ
และความเสี่ยง 8 ข้อที่มาจากสภาพจริงของรีโปนี้

เอกสารนี้อธิบายว่า “สัญญา” (contract) ระหว่าง frontend กับ backend ในสถาปัตยกรรมแบบ tRPC คืออะไรกันแน่ ต้องประกอบด้วยอะไรบ้าง และเมื่อเอามาวางบน กองโค้ดของทีม ULMS มีอยู่จริงตอนนี้ จะเจอข้อจำกัดอะไรที่ต้องตัดสินใจก่อน

รูปทั้งหมดเป็นภาพอธิบายที่วาดขึ้นเพื่อสอน **ไม่ใช่ผลการทดลอง** แต่ข้อเท็จจริง ทุกข้อ (เวอร์ชันไลบรารี จำนวนตาราง สถานะการตั้งค่า) สืบกลับไปยังไฟล์จริงในรีโปได้ ตามรายการในภาคผนวก ง

ภาค 5 คือลำดับงานที่เสนอให้ทำ และเอกสารคู่กันอีกฉบับ (trpc-meeting.tex) คือวาระประชุมสำหรับตกลงสิ่งที่เอกสารนี้ชี้ว่าต้องตกลง

สารบัญ

เอกสารนี้อ่านอย่างไร	4
1 tRPC คืออะไร และสัญญาของมันอยู่ตรงไหน	6
1 ปัญหาที่ tRPC เกิดมาเพื่อแก้	6
2 สัญญาของ tRPC คืออะไรกันแน่	7
2.1 ผลที่ 1 — สัญญามีผลเฉพาะตอน compile เท่านั้น	7
2.2 ผลที่ 2 — สัญญาต้องเดินทางข้ามฝั่งให้ได้จริง	7
2.3 ผลที่ 3 — “เอกสาร API” ไม่เกิดขึ้นเองอัตโนมัติ	8
3 ขอบเขต — tRPC ไม่ใช่อะไรบ้าง	8
2 ทางเดินของ request หนึ่งครั้ง	9
4 ขั้นที่ 1-2 — จาก component ถึง TanStack Query	9
5 ขั้นที่ 3-4 — tRPC client และ link	10
6 ขั้นที่ 5 — ระหว่างทาง: สิ่งที่ JSON ทำหาย	10
7 ขั้นที่ 6-7 — context และ middleware	11
8 ขั้นที่ 8 — zod ตรวจสอบ input	12
9 ขั้นที่ 9-10 — service, Prisma และด้านแปลงขาออก	12
9.1 output ต้องบรรจุ “ค่าที่คำนวณแล้ว” ไม่ใช่วัตถุดิบให้ไปคำนวณเอง	13
3 ต้องมีอะไรบ้างให้ครบ	15
10 ฝั่ง backend — ห้าขั้นที่ขาดไม่ได้	15
10.1 หน้าตาของ router ที่จะเขียน	15
11 ฝั่ง frontend — สามขั้นที่ยังไม่มี	16
12 ตัวเชื่อม — ขั้นที่สำคัญที่สุดและยังไม่มีเลย	17

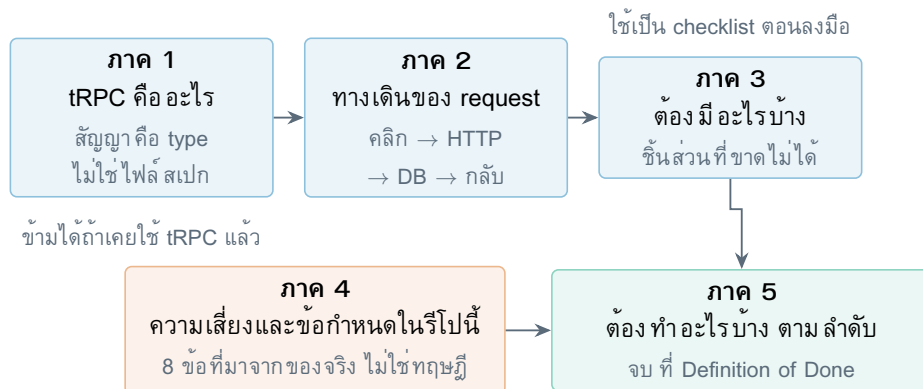
13 checklist รวม	17
4 ความเสี่ยงและข้อกำหนดในรีโปนี้	18
14 ความเสี่ยง 1 — zod คนละเวอร์ชัน	18
15 ความเสี่ยง 2 — คนละโปรเจกต์ npm คนละสาขา git	19
16 ความเสี่ยง 3 — schema เป็น PascalCase และไม่มี enum เลย	20
17 ความเสี่ยง 4 — DateTime กลายเป็นสตริง	20
18 ความเสี่ยง 5 — อัปเดตรูปผ่าน tRPC ไม่ได้	21
19 ความเสี่ยง 6 — polling ถี่ปะทะ batching	21
20 ความเสี่ยง 7 — ยังไม่ได้เปิด CORS และยังไม่มีเส้นทางของ cookie	22
21 ความเสี่ยง 8 — ลืมใส่ output แล้วได้ any	23
5 ต้องทำอะไรบ้าง ตามลำดับ	24
22 ลำดับงานที่เสนอ	24
22.1 ทำไมต้องทำเส้นเดียวให้ครบก่อน	24
22.2 stub ที่คืนค่า mock — ประโยชน์ที่แท้จริงของการทำสัญญาก่อน	25
23 กติกาที่ต้องบังคับ	25
24 Definition of Done ของ procedure หนึ่งตัว	25
ภาคผนวก	27
ภาคผนวก	27
ก อภิธานศัพท์ ไทย-อังกฤษ	27
ข ร่องตารางสัญญาที่ต้องทำให้เสร็จ	28
ค คำสั่งที่ต้องรันจริง	29

ง ที่มาของข้อเท็จจริงทุกข้อในเอกสารนี้

29

เอกสารนี้อ่านอย่างไร

เอกสารแบ่งเป็นห้าภาค เรียงตามเส้นทางที่ request หนึ่งครั้งเดินทางจริง ไม่ได้เรียงตามหัวข้อทฤษฎี เหตุผลคือเมื่อผู้อ่านหลงว่าตัวเองอยู่ตรงไหน คำถามที่ตอบ ตัวเองได้เสมอคือ “ตอนนี้ข้อมูลอยู่ในรูปอะไร และอยู่ที่เครื่องไหน” — การเรียงตาม หัวข้อทฤษฎีไม่ให้หลักยึดแบบนั้น



รูปที่ 1: แผนที่ของเอกสาร ภาค 1-3 เป็นความรู้ที่ย้ายไปโครงการอื่นได้ ส่วนภาค 4-5 ผูกกับรีโปนี้โดยเฉพาะ

ระบบสีในเอกสารฉบับนี้

สีไม่ได้ใส่เพื่อความสวย ในเล่มนี้สีสี่สีแทน **สี่ชั้นของระบบ** และใช้ความหมาย เดียวกันทุกที่ ทั้งในเนื้อความ ในไต่อะแกรม และในตาราง เมื่อผู้อ่านเห็นกล่องสีส้ม ในรูปที่หน้า 20 จะรู้ทันทีว่ามันคือของฝั่ง backend โดยไม่ต้องอ่านป้าย

- **สัญญา** — สิ่งที่เป็น “สัญญา” เอง: zod schema, type ของ AppRouter, input/output
- **backend** — ฝั่ง NestJS: router, service, Prisma, ฐานข้อมูล
- **frontend** — ฝั่ง React: component, hook, TanStack Query cache
- **เครือข่าย** — ระหว่างทาง: HTTP, cookie, การแปลงเป็น JSON, การรวม request

ชุดสีฐานคือ Okabe-Ito ซึ่งเลือกมาให้แยกออกจากกันได้แม้ผู้อ่านมองสีบางคู่ไม่ต่าง

กล่องสีแบบ

กล่องแต่ละสีมีความหมายตายตัวทั้งเล่ม ตัวอย่างข้างล่างนี้ คือ คำนิยามของมัน

ฟ้า — ใจความที่ต้องจำติดตัวไป

ใช้กับข้อสรุปที่ร้อยหลายบทเข้าด้วยกัน ถ้าอ่านเอกสารนี้แล้วจำได้แค่กล่องฟ้า ก็ถือว่าได้สาระหลักไปแล้ว

ส้ม — กับดัก หรือข้อสมมติที่ถ้าลืมแล้วข้อสรุปจะผิด

ใช้กับจุดที่คนพลาดกันบ่อย และกับข้อจำกัดที่ทำให้ข้อสรุปในหน้าเดียวกันใช้ไม่ได้ ถ้าเงื่อนไขไม่ครบ

เขียว — ตรวจสอบกับของจริงในรีโปแล้ว

ใช้เมื่อข้อความนั้นได้จากการเปิดไฟล์จริงดู ไม่ใช่จากความจำหรือจากเอกสารของไลบรารีใช้อย่างประหยัด

แดง — วิธีที่ผิดจริง ๆ

ใช้กับแนวทางที่ดูสมเหตุสมผลแต่จะพังในภายหลัง พร้อมเหตุผลว่าพังเพราะอะไร เป็นกล่องที่ควรมีน้อยที่สุด

ภาค 1

tRPC คืออะไร และสัญญาของมันอยู่ตรงไหน

คำถามที่ภาคนี้ตอบ: ก่อนจะลงมือเขียน เราตกลงอะไรกันแน่ และไฟล์ “สัญญา” ที่ว่านั้นหน้าตาเป็นอย่างไร

1 ปัญหาที่ tRPC เกิดมาเพื่อแก้

ในระบบที่ frontend กับ backend เขียนแยกกัน ปัญหาที่กินเวลาที่มากที่สุด ไม่ใช่การเขียน endpoint แต่คือ ความจริงสองชุดที่ค่อย ๆ เลื่อนออกจากกัน (type drift)

ตอนนี้ในรีโปของเราเรามีการนั้นครบทุกองค์ประกอบแล้ว ทั้งที่ยังไม่ได้ต่อ API กันเลยด้วยซ้ำ



คอม เมนต์ บรรทัด แรก ของ frontend/ src/ types/ domain.ts เขียนไว้เองว่า “ตรง กับ schema ฝั่ง backend (Prisma) — sync กัน เป็น ระเบียบ”

ประโยคนี้คือคำจำกัดความของหนี้ทางเทคนิค: ความถูกต้อง ขึ้นกับวินัยของคน ไม่ใช่ขึ้นกับเครื่องมือ

รูปที่ 2: สภาพปัจจุบันของรีโป — ความจริงสองชุดที่ยังไม่มีสะพานเชื่อม นี่คือช่องว่างที่ tRPC เข้ามาปิด

วิธีแก้แบบคลาสสิกมีสามแนว และแต่ละแนวจ่ายราคาคนละแบบ

แนวทาง	สัญญาอยู่ที่ไหน	ได้อะไร	จ่ายอะไร
REST + เขียน type เอง	ไม่มีที่ไหน (อยู่ในหัวคน)	เริ่มเร็วที่สุด เข้าภาษาได้	type เพี้ยนเงิบ ๆ รู้ตอน runtime
OpenAPI / codegen	ไฟล์ openapi.yaml	เข้าภาษาได้ มีเอกสารอัตโนมัติ	ต้องรัน codegen ทุกครั้ง เครื่องมือหนัก
GraphQL	schema SDL	client เลือกฟิลด์เองได้	ต้องเรียนของใหม่ทั้งทีม N+1 ต้องระวัง
tRPC	type ของ TypeScript เอง	ผิดปุ๊บ compile ไม่ผ่านปั๊บ ไม่ต้องเรียนภาษาใหม่	ผูกกับ TypeScript ทั้งสองฝั่งเท่านั้น

ตารางที่ 1: สี่แนวทาง ต่างกันที่ “สัญญาถูกเก็บไว้ที่ไหน” — และนั่นเป็นตัวกำหนดว่าใครเป็นคนจับผิด

ทำไม tRPC จึงเหมาะกับโครงการนี้

ทีมนี้ใช้ TypeScript ทั้งสองฝั่งอยู่แล้ว (Vite + React กับ NestJS) และเป็นทีมนักศึกษาที่มีเวลาจำกัด — ประโยชน์ข้ามภาษาของ OpenAPI/GraphQL จึงไม่ได้ใช้ ในขณะที่ต้นทุนการเรียนรู้ของมันต้องจ่ายเต็ม tRPC ให้สิ่งที่ต้องการที่สุด (จับ type ผิดตั้งแต่ตอน compile) ด้วยของที่ทีมรู้จักอยู่แล้ว

2 สัญญาของ tRPC คืออะไรกันแน่

นี่คือจุดที่คนเข้าใจผิดมากที่สุด และเข้าใจผิดแล้วจะวางแผนงานผิดตาม

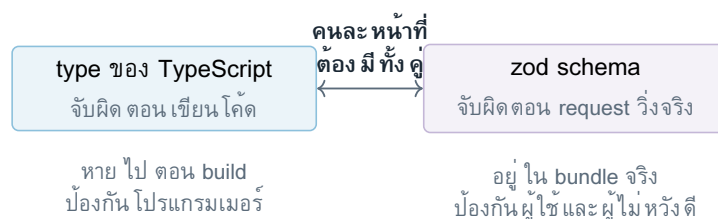
tRPC ไม่มีไฟล์สเปกกลาง — สัญญาคือ type ของ TypeScript

ใน OpenAPI สัญญาคือไฟล์ .yaml ที่คนอ่านได้ใน gRPC คือไฟล์ .proto แต่ใน tRPC สัญญาคือ ชนิดข้อมูลของตัวแปรชื่อ **AppRouter** ที่ backend export ออกมา แล้ว frontend import type เข้าไป จากนั้น TypeScript จะอนุมาน input และ output ของทุก procedure ให้เอง ถ้า backend เปลี่ยนชื่อฟิลด์ frontend จะ **compile** ไม่ผ่านทันที — ไม่ต้องรัน ไม่ต้องเปิดเบราว์เซอร์

ผลที่ตามมาซึ่งต้องเข้าใจให้ตรงกันทั้งทีมมีสามข้อ

2.1 ผลที่ 1 — สัญญามีผลเฉพาะตอน compile เท่านั้น

type ของ TypeScript หายไปหมดตอน build เป็น JavaScript แปลว่าถ้ามีใครยิง request เข้ามาตรง ๆ ด้วย ข้อมูลผิดรูป type ช่วยอะไรไม่ได้เลย สิ่งป้องกันจริงตอน runtime คือ **zod schema** ต่างหาก



รูปที่ 3: type กับ zod ไม่ใช่ของซ้ำซ้อนกัน — ตัวหนึ่งทำงานก่อนโค้ดรัน อีกตัวทำงานตอนโค้ดรันแล้ว

อย่าคิดว่า “มี type แล้วไม่ต้อง validate”

input ทุกตัวต้องมี zod schema เสมอ แม้ frontend ของเราเองจะส่งค่าถูก เพราะ endpoint เปิดรับจากอินเทอร์เน็ต ใครก็ยิงเข้ามาได้ — ระบบนี้มีเรื่อง เครดิตผู้ใช้และการอนุมัติของอาจารย์อยู่ในนั้น

2.2 ผลที่ 2 — สัญญาต้องเดินทางข้ามฝั่งให้ได้จริง

ถ้าสัญญาคือ type แปลว่า frontend ต้อง “มองเห็น” ฟิลด์ของ backend ผ่านระบบ module ของ TypeScript ให้ได้ ซึ่งในรีโปนี้ยัง **ทำไม่ได้** เพราะ frontend/ กับ backend-preview/backend/ เป็นคนละโปรเจกต์

กต์ npm และอยู่คนละสาขา git — รายละเอียดอยู่ในภาค 4 หัวข้อ 15 และเป็นวาระที่ต้องตกลงเป็นอันดับแรกสุด

2.3 ผลที่ 3 — “เอกสาร API” ไม่เกิดขึ้นเองอัตโนมัติ

OpenAPI แถมหน้าเอกสารให้ฟรี แต่ tRPC ไม่มี สิ่งทดแทนได้คือ ตารางสัญญา ที่ทีมเขียนเอง (ดูภาคผนวก ข) ซึ่งไม่ใช้งานเสียเปล่า เพราะ ตารางนั้นใช้เป็นภาคผนวกของรายงานส่งอาจารย์ได้ด้วย

3 ขอบเขต — tRPC ไม่ใช่อะไรบ้าง

การรู้ขอบเขตช่วยกันไม่ให้ทีมพยายามยึดทุกอย่างเข้าไปในเครื่องมือเดียว

ความเข้าใจผิด	ความจริง
“tRPC แทน REST ได้ทั้งหมด”	แทนได้เกือบหมด ยกเว้นการอัปโหลดไฟล์ เพราะ tRPC คุยกันด้วย JSON ส่ง binary ไม่ได้ — โครงการนี้มีอัปโหลดรูปตอนรับและตอนคืนของ จึงต้องเหลือทาง REST ไว้
“tRPC เป็นโปรโตคอลใหม่”	ข้างล่างมันคือ HTTP POST/GET ธรรมดา เปิด Network tab ก็เห็นแค่รูปแบบ URL และ body ถูกกำหนดโดยไลบรารี
“ใช้ tRPC แล้วภาษาอื่นเรียกได้”	เรียกได้ในทางเทคนิค แต่จะไม่ได้ประโยชน์อะไรเลยเพราะสัญญาอยู่ในรูป type ของ TypeScript ถ้าอนาคตต้องมีแอปมือถือที่ไม่ใช่ TypeScript ต้องวางแผนใหม่
“tRPC จัดการ cache ให้”	ไม่ — คนจัดการ cache, refetch, polling คือ TanStack Query ซึ่งรีโปนี้ลงไว้แล้ว (@tanstack/react-query รุ่น 5) tRPC แคเป็นตัวยิงและตัวให้ type

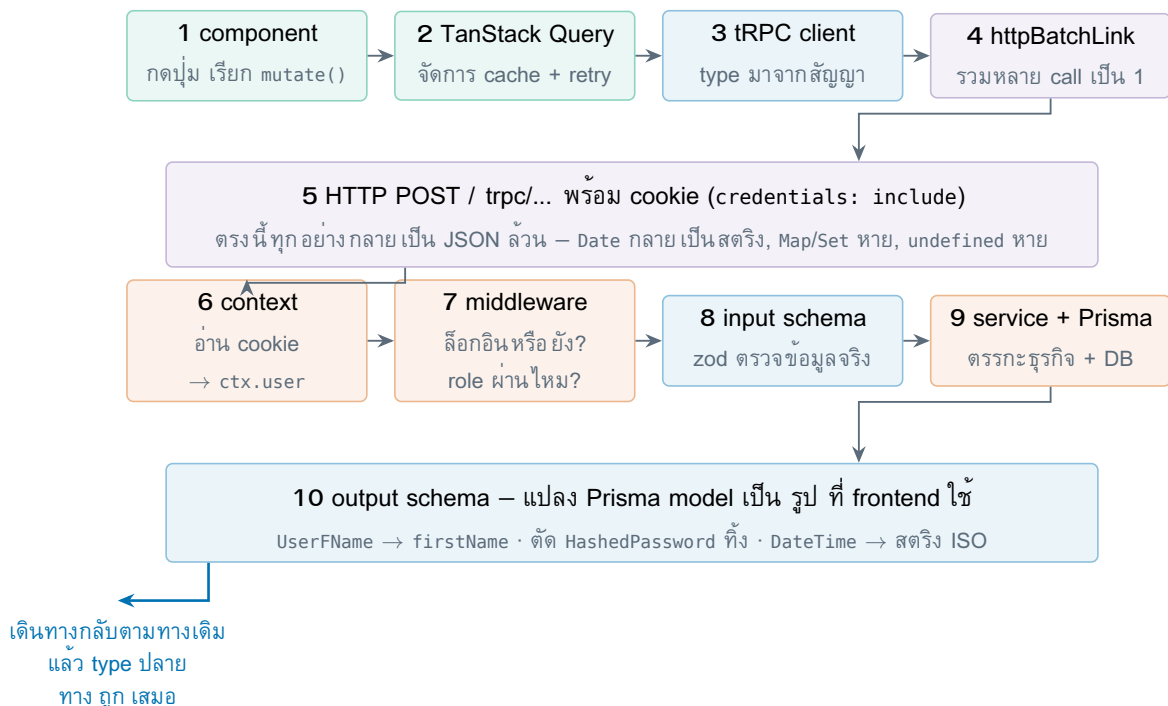
ตารางที่ 2: ขอบเขตของ tRPC — ข้อแรกกับข้อสุดท้ายเป็นข้อที่กระทบแผนงานของทีมโดยตรง

ภาค 2

ทางเดินของ request หนึ่งครั้ง

คำถามที่ภาคนี้ตอบ: ตั้งแต่ผู้ใช้กดปุ่ม “ส่งคำขอยืม” จนข้อมูลกลับมาแสดงบนจอ มันผ่านมือใครบ้าง และแต่ละมือทำอะไรกับข้อมูล

ภาคนี้คือแกนกลางของเอกสาร ถ้าเข้าใจรูปที่ 4 ได้ทั้งรูป ที่เหลือเป็นรายละเอียด



รูปที่ 4: ทางเดินของ request หนึ่งครั้ง สืบบอกว่าขั้นนั้นเป็นความรับผิดชอบของใคร — ขั้นที่เป็นสีฟ้า (3, 8, 10) คือ “สัญญา” และเป็นสามจุดเดียวที่ทั้งสองฝั่งต้องตกลงกันให้ตรง

4 ขั้นที่ 1-2 — จาก component ถึง TanStack Query

ฝั่ง React จะไม่ได้เรียก fetch ตรง ๆ อีกต่อไป แต่เรียกผ่าน hook ที่ tRPC สร้างให้ ซึ่งข้างในก็คือ TanStack Query ที่ทีมใช้อยู่แล้ว

```
// what a screen looks like after the contract exists
const { data, isLoading } = trpc.item.list.useQuery({
  page: 1,
  q: searchText,
});
// `data` is fully typed. No `as Equipment[]`, no hand-written interface.

const createLoan = trpc.loan.create.useMutation({
```

```
onSuccess: () => {
  // refetch the lists this action invalidates
  utils.loans.myActive.invalidate();
  utils.items.list.invalidate();
},
});
```

สังเกตว่างานที่ TanStack Query รับไปทำคือสิ่งที่เอกสาร “รายการเรียกใช้งานจาก Backend” ระบุไว้ทั้งกลุ่มที่ 1 และกลุ่มที่ 2 พอตี: polling ทุก 10–15 วินาที คือ `refetchInterval` และ “refetch หลังทำ action” คือ `invalidate()`

แบ่งงานให้ชัด: tRPC ยิงกับให้ type · TanStack Query จำและตัดสินใจว่าจะยิงเมื่อไร

ถ้าคำถามคือ “ข้อมูลนี้เก่าไปหรือยัง” “ต้อง refetch ใหม่” “มีคนถามพร้อมกันสองที่ จะยิงซ้ำไหม” — คำตอบทั้งหมดอยู่ที่ TanStack Query ไม่ใช่ tRPC เวลาตีบักให้แยกสองเรื่องนี้ออกจากกันก่อนเสมอ

5 ขั้นที่ 3–4 — tRPC client และ link

link คือขั้นที่ตัดสินใจว่า “จะเอา call นี้ส่งออกไปยัง” ตัวที่จะใช้คือ `httpBatchLink` ซึ่งมีพฤติกรรมหนึ่งที่ต้องรูล่วงหน้า ไม่งั้นจะงงตอนตีบัก

`httpBatchLink` รวมหลาย query เป็น request เดียว — Network tab จะไม่ตรงกับที่คิด

ถ้าหน้าหนึ่งเรียก 4 query พร้อมกัน คุณจะเห็นใน DevTools แค่ **หนึ่งบรรทัด** ที่มี URL หน้าตาประหลาดแบบ `/trpc/item.list,user.me,credit.mine,notification.unread?batch=1` ผลตามมาที่ต้องระวังสองข้อ

1. **ช้าที่สุดลากทุกตัว** — response ทั้งก่อนกลับพร้อมกัน ถ้ามี query เดียวที่ช้า 3 วินาที อีกสามตัวก็รอดวย
 2. **หน้าที่ polling ถีต้องระวังเป็นพิเศษ** — เอกสารระบุ polling 10–15 วินาทีสำหรับของคงเหลือและสล็อตห้อย ถ้าไปรวมก่อนกับ query หนักอย่างรายงานสถิติ ทั้งก่อนจะช้าตามทุก 10 วินาที
- ถ้าเจอปัญหานี้ ทางแก้คือแยก link ตามเงื่อนไข (`splitLink`) หรือใช้ `httpLink` ธรรมดาสำหรับเส้นที่ถี่

6 ขั้นที่ 5 — ระหว่างทาง: สิ่ง JSON ทำหาย

จุดนี้เป็นบ่อเกิดของบั๊กที่หายากที่สุดในระบบแบบนี้ เพราะ type ผัง TypeScript บอกว่าเป็น `Date` แต่ของที่มาถึงจริงเป็นสตริง

ส่งออกจาก backend	ถึง frontend เป็น	ผลกับโครงการนี้
Date	สตริง ISO	กระทบหนัก — schema.prisma มีฟิลด์ DateTime อย่างน้อย 19 จุด เช่น DueTime, CheckoutTime, ExpirationTime
undefined	หายไปจาก object	ต้องระวังตอนเช็ค 'key' in obj
Decimal (Prisma)	สตริง หรือ ฟัง	ยังไม่กระทบ — schema ปัจจุบันไม่ใช่ Decimal
Map, Set, BigInt	ฟัง	อย่าใส่ใน output

ตารางที่ 3: การแปลงเป็น JSON คือด้านที่ข้อมูลเปลี่ยนรูปโดยที่ type ไม่บอก

มีสองทางเลือกและต้องเลือกให้เหมือนกันทั้งระบบ

ทางเลือก	ได้อะไร	เสียอะไร
ใส่ superjson เป็น transformer	ได้ Date จริงกลับมา เขียนโค้ดฝั่งหน้าเว็บง่ายกว่า	ต้องตั้งค่าให้ตรงกันเป๊ะทั้งสองฝั่ง ถ้าตั้งข้างเดียวจะพังแบบอ่านสาเหตุไม่ออก เพิ่มขนาด payload
ประกาศเป็นสตริง ISO ใน output schema	ตรงไปตรงมา ดีบั๊กง่าย เห็นใน Network tab แล้วเข้าใจทันที ที่มี date-fns อยู่แล้ว	ต้อง parseISO เองทุกที่ที่จะเอาไปคำนวณ

ตารางที่ 4: ข้อเสนอของเอกสารนี้คือแถวที่สอง — แต่เป็นเรื่องที่ต้องมีมติร่วมกัน ไม่ใช่ใครเลือกเองกลางทาง

7 ขั้นที่ 6-7 — context และ middleware

context คือ object ที่ถูกสร้างใหม่ทุก request และส่งต่อให้ทุก procedure ในคำขอนั้น หน้าที่ของมันคือแปลง “สิ่งที่มากับ HTTP” ให้เป็น “สิ่งที่ตรรกะธุรกิจใช้ได้”



รูปร่างของ ctx.user คือสิ่งที่ต้องตกลง ก่อน เขียน procedure ตัวแรก เพราะทุก procedure พึ่งมัน ถ้า ลืม ใส่ facultyKey ไว้ ตั้งแต่ ต้น แล้ว วัน หลัง พบ ว่าการ ตรวจ สิทธิ์ ยืม ต้อง ใช้ ต้อง กลับ ไป แก้ ทุก procedure ที่ เขียน ไป แล้ว

รูปที่ 5: context คือด้านแปลงจากภาษาเครือข่ายเป็นภาษาธุรกิจ — และเป็นสัญญาภายในที่ทุก procedure ใช้ร่วมกัน

middleware คือขั้นที่คั่นก่อนเข้า procedure ใช้บังคับเรื่องสิทธิ์ รูปแบบที่ควรใช้คือสร้าง procedure สำเร็จรูปไว้เป็นขั้น ๆ แล้วเลือกใช้ตามงาน แทนที่จะเขียน if ตรวจ role ซ้ำในทุก service

```
// four ready-made procedure levels, defined once
publicProcedure    // anyone: login, catalog browsing
protectedProcedure // any logged-in user
staffProcedure     // staff only: prepare, check-in, damage assessment
supervisorProcedure // supervisor only: approve, rule on appeals
adminProcedure     // admin only: user management, system settings
```

อย่าตรวจ role ข้างใน service

if (user.role !== 'staff') throw ... ที่กระจายอยู่ใน service หลายสิบไฟล์ ทำให้ตอบคำถามว่า “ใครเรียก procedure นี้ได้บ้าง” ไม่ได้ โดยไม่อ่านโค้ดทั้งไฟล์ — และนั่นคือคำถามที่ต้องตอบได้จากตารางสัญญาสิทธิ์การเข้าถึง เป็นส่วนหนึ่งของสัญญา ไม่ใช่รายละเอียดการทำงานภายใน

8 ขั้นที่ 8 — zod ตรวจ input

ด้านนี้คือสิ่งเดียวที่ป้องกันข้อมูลผิดรูปได้จริงตอนรัน ถ้า schema ไม่ผ่าน tRPC จะโยน BAD_REQUEST กลับไปโดยที่ service ไม่ถูกเรียกเลย

```
export const createLoanInput = z.object({
  itemId: z.number().int().positive(),
  startDate: z.string().datetime(), // ISO string, per the team decision
  endDate: z.string().datetime(),
  note: z.string().max(500).optional(),
});
export type CreateLoanInput = z.infer<typeof createLoanInput>;
```

เขียน schema หนึ่งครั้ง ได้ทั้ง validation และ type

z.infer ดึง type ของ TypeScript ออกมาจาก schema เดียวกัน แปลว่ากฎการตรวจสอบกับ type จะไม่มีวันไม่ตรงกัน — นี่คือเหตุผลที่ tRPC เลือกผูกกับ zod ตั้งแต่แรก

9 ขั้นที่ 9-10 — service, Prisma และด้านแปลงขาออก

ขั้นที่ 10 เป็นขั้นที่ทีมมักข้าม และเป็นข้อผิดพลาดที่แพงที่สุดในระยะยาว

ห้าม return Prisma model ตรง ๆ ออกไปเป็น output

มันดูสะดวกมาก เพราะ type ก็มีให้พร้อมอยู่แล้ว แต่มีปัญหาสามข้อที่ร้ายแรงกับ โครงการนี้โดยเฉพาะ

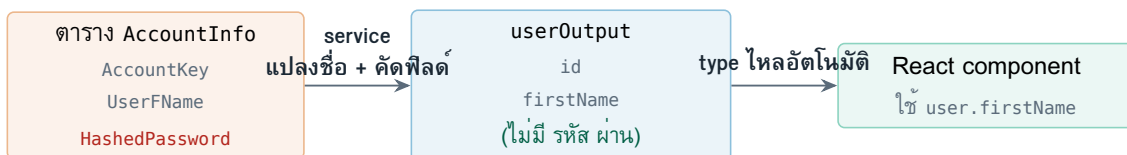
1. ข้อมูลลับหลุด — AccountInfo มีฟิลด์ HashedPassword อยู่ในตารางเดียวกับ ชื่อ และ อีเมล findUnique แล้วส่งกลับตรง ๆ คือส่ง hash รหัสผ่านออกไปที่เบราว์เซอร์
2. ฐานข้อมูลเปลี่ยนไม่ได้อีกเลย — ถ้าสัญญาคือรูปร่างของตาราง การเปลี่ยนชื่อคอลัมน์จะทำให้

frontend พังทันที ทั้งที่ไม่ควรเกี่ยวกัน

3. frontend ต้องพูดภาษาฐานข้อมูล — schema ของเราใช้ PascalCase (UserName, UserCredit, AccountKey) ซึ่งขัดกับธรรมเนียมของโค้ด React ทั้งโปรเจกต์

ทางที่ถูกคือให้ output schema เป็น “ภาษาของ frontend” แล้วให้ service เป็นคนแปลง ขั้นตอนการแปลงนี้เล็กน้อยแต่เป็นสิ่งที่ทำให้สองฝั่งเป็นอิสระต่อกัน

```
export const userOutput = z.object({
  id: z.number().int(), // from AccountKey
  studentId: z.string(), // from UserID
  firstName: z.string(), // from UserName
  lastName: z.string(), // from UserLName
  email: z.string().email(),
  role: z.enum(['borrower', 'staff', 'supervisor', 'admin']),
  creditScore: z.number().int(), // from UserCredit
  // HashedPassword is not here, and cannot leak by accident
});
```



ชั้น ตรง กลาง นี้ ราคา ไม่ แพง แต่ ชื่อ อีสระภาพ สอง ทาง: DBA เปลี่ยน ชื่อ คอลัมน์ ได้ โดย ไม่ พัง หน้า เว็บ และ คน ทำ frontend ไม่ ต้อง รู้ ว่า ฐาน ข้อมูล ตั้ง ชื่อ ยัง ไง

ใน รี โป นี้ ชั้น นี้ จำเป็น กว่า ปกติ เพราะ schema ใช้ PascalCase และ ไม่มี enum เลย สัก ตัว (สถานะ ทั้งหมด เป็น ตาราง แยก)

รูปที่ 6: ด่านแปลงขาออก — จุดที่ “รูปร่างของฐานข้อมูล” กับ “รูปร่างของสัญญา” แยกออกจากกันอย่างตั้งใจ

9.1 output ต้องบรรจุ “ค่าที่คำนวณแล้ว” ไม่ใช่วัตถุดิบให้ไปคำนวณเอง

กฎเครดิตของระบบนี้เป็นตัวอย่างที่ดีที่สุดของหลักข้อนี้ และเป็นจุดที่เข้าใจผิดกันบ่อย

เครดิตต่ำไม่ได้ทำให้ยืมไม่ได้ — มันทำให้ยืมได้สั้นลง

ใน schema.prisma คะแนนเครดิตถูกจับเป็นระดับผ่าน CreditTier (CreditMin, CreditMax) แล้วระดับนั้นไปคู่กับกฎการยืมในตาราง BorrowConstraints ซึ่งเก็บ MaxBorrowDate (จำนวนวันยืมสูงสุด) และ MaxExtendTime (จำนวนครั้งที่ต่ออายุได้)

ไม่มีฟิลด์ไหนในสคิมาที่บอกว่าคะแนนเท่าไรถึงห้ามยืม สิ่งที่กันไม่ให้ยืม เป็นคนละกลไก คือตาราง Eligibility (สังกัด × บทบาท × ของ) และ MinimumAuthorityLevel — ทั้งคู่ไม่เกี่ยวกับคะแนนเครดิต

ฝั่ง frontend ก็มองแบบเดียวกันอยู่แล้ว: CreditBandConfig ใน types/domain.ts มีฟิลด์ loanDays ไม่ใช่ฟิลด์ “ยืมได้/ไม่ได้”

ผลต่อการออกแบบ output มีสองข้อ

1. `loan.create` ต้อง คำนวณครบกำหนดจริง ที่ระบบให้ ไม่ใช่แค่ “สำเร็จ” แล้วให้ frontend เอาจำนวนวันของระดับเครดิตไปบวกเอง เพราะ ถ้าเครดิตถูกหักระหว่างทาง สองฝั่งจะคำนวณได้คนละคำตอบ
2. `auth.me` หรือ `credit.mine` ควรคืน เพดานวันยืมและ จำนวนครั้งต่ออายุที่เหลือ มาเป็นค่าสำเร็จรูป ไม่ใช่คืนคะแนนดิบแล้วให้ frontend ไปเปิดตารางเทียบเอง — ตารางเทียบนั้นคือกฎธุรกิจ และกฎธุรกิจอยู่ฝั่ง backend ที่เดียว

ถ้า frontend คำนวณเองเมื่อไร กฎจะมีสองชุดทันที

วันที่ทีมเปลี่ยนเพดานของระดับ D2 จาก 7 วันเป็น 5 วัน ถ้ากฎถูกคัดลอกไว้ฝั่ง frontend ด้วย จะมีคนแก้ที่เดียวแล้วอีกฝั่งค้าง — และเป็นความไม่ตรงกันที่ type ตรวจไม่เจอ เพราะทั้งสองฝั่งยังเป็น number เหมือนเดิม

ภาค 3

ต้องมีอะไรบ้างให้ครบ

คำถามที่ภาคนี้ตอบ: ถ้าจะบอกว่า “เรามีสัญญา tRPC แล้ว” ต้องมีชิ้นส่วน อะไรอยู่ในรีโปบ้าง และตอนนี้ขาดอะไร

10 ฝั่ง backend — ห้าชั้นที่ขาดไม่ได้

รีโปนี้ใช้ nestjs-trpc ซึ่งทำงานต่างจาก tRPC เพียง ๆ ตรงที่มันเป็นแบบ เขียนโค้ดก่อนแล้วให้เครื่องสร้างไฟล์สัญญาให้ (code-first + codegen) แทนที่จะประกอบ router ด้วยมือ เราติด decorator บน class แล้วมันจะสแกนโค้ดตอน build เพื่อ เขียนไฟล์ router ออกมาให้ — ไฟล์นั้นแหละคือตัวสัญญาที่ frontend ต้องเอาไปใช้

ตรวจสอบกับรีโปแล้ว: ของที่ลงไว้ครบ แต่ยังไม่ได้เปิดใช้

backend-preview/backend/package.json มี nestjs-trpc รุ่น 2.13, @trpc/server รุ่น 11.18, zod รุ่น 4.4 และ @prisma/client รุ่น 7.9 ครบแล้ว

แต่ src/app.module.ts เรียก TRPCModule.forRoot() แบบ ไม่ส่งค่าตั้งค่าใด ๆ แปลว่ายังไม่ได้บอกให้มันสร้างไฟล์สัญญา และใน src/ ยังไม่มีไฟล์ router สักไฟล์เดียว

#	ชิ้นส่วน	หน้าที่	สถานะในรีโป
1	TRPCModule + autoSchemaFile	บอกให้ Nest เปิด endpoint และให้เขียนไฟล์สัญญาออกมา	เรียกแบบไม่ส่ง option
2	context factory	สร้าง ctx.user จาก cookie ทุก request	ยังไม่มี
3	middleware ตาม role	บังคับสิทธิ์ก่อนเข้า service	ยังไม่มี
4	router ต่อโดเมน	item, loan, reservation, appeal, credit, notification, admin	ยังไม่มีสักตัว
5	input/output schema	ตรวจสอบข้อมูลขาเข้า และกำหนดรูปร่างขาออก	ยังไม่มี

ตารางที่ 5: ชิ้นส่วนฝั่ง backend — ทั้งห้าชั้นยังไม่มี แต่ของที่ต้องลงเพิ่มคือศูนย์ ทุกอย่างติดตั้งไว้แล้ว

10.1 หน้าตาของ router ที่จะเขียน

```
@Router({ alias: 'loan' })
export class LoanRouter {
  constructor(private readonly loanService: LoanService) {}
```



```

@UseMiddlewares(AuthMiddleware)
@Query({ input: listLoansInput, output: listLoansOutput })
list(@Input() input: ListLoansInput, @Ctx() ctx: TrpcContext) {
  return this.loanService.listForUser(ctx.user.id, input);
}

@UseMiddlewares(AuthMiddleware)
@Mutation({ input: createLoanInput, output: loanOutput })
create(@Input() input: CreateLoanInput, @Ctx() ctx: TrpcContext) {
  return this.loanService.create(ctx.user.id, input);
}
}

```

nestjs-trpc ต้องระบุ output เอง ไม่งั้นสัญญาไร้ความหมาย

tRPC เพียว ๆ อนุมาน output จากค่าที่ฟังก์ชัน return ได้เอง แต่ nestjs-trpc สร้างไฟล์สัญญาจากการอ่านโค้ดแบบสถิต มันจึงต้องมี schema ที่เขียนไว้ตรง ๆ ให้อ่าน ถ้าลืมใส่ output ฝั่ง frontend จะได้ type เป็น any แล้วทุกอย่างจะ “compile ผ่าน” โดยไม่มีการตรวจสอบใด ๆ เหลืออยู่เลย — เป็นความล้มเหลวที่เจ็บที่สุดที่เป็นไปได้ในสถาปัตยกรรมนี้

ข้อนี้ต้องเป็นกฎของทีมและต้องมีคนตรวจตอนรีวิว PR เพราะ compiler จับให้ไม่ได้ — ดูข้อเสนอวิธีบังคับในหัวข้อ 21

11 ฝั่ง frontend — สามขั้นที่ยังไม่มี

ตรวจสอบกับรีโปแล้ว: frontend/package.json ยังไม่มีแพ็คเกจของ tRPC เลย

มี @tanstack/react-query รุ่น 5.59 และ zod รุ่น 3.23 แต่ไม่มี @trpc/client และไม่มี @trpc/react-query — และเวอร์ชัน zod ก็ไม่ตรงกับฝั่ง backend ซึ่งเป็นความเสี่ยงข้อแรกในภาค 4

1. แพ็คเกจ — @trpc/client และตัวเชื่อมกับ TanStack Query ต้องเลือกเวอร์ชันให้เข้าชุดกับ @trpc/server รุ่น 11 ที่ backend ใช้
2. ตัวตั้งค่า client — กำหนด link, URL และที่สำคัญคือ credentials: 'include' เพื่อให้ cookie ติดไปด้วย
3. ไฟล์สัญญาที่นำเข้ามาได้ — ข้อนี้คือข้อที่ยังไม่มีทางทำ ดูภาค 4

```

// frontend/src/lib/trpc.ts - the shape this file will take
import { createTRPCClient, httpBatchLink } from '@trpc/client';
import type { AppRouter } from '@server-types/appRouter'; // the contract

export const trpcClient = createTRPCClient<AppRouter>({
  links: [
    httpBatchLink({
      url: import.meta.env.VITE_API_URL + '/trpc',
      // the team already chose httpOnly cookies, so every call must carry them
      fetch: (url, opts) => fetch(url, { ...opts, credentials: 'include' }),
    })
  ]
});

```

```
    },
    ],
  });
```

api-client.ts เติมไม่ได้ทั้ง

ไฟล์ frontend/src/lib/api-client.ts ที่มีอยู่ยังต้องใช้สำหรับ uploadFile เพราะ tRPC ส่งไฟล์ไม่ได้ ส่วนเมทอด get/post ทั่วไปจะถูกแทนที่ด้วย tRPC ที่ละ feature — ไม่ต้องรื้อทีเดียวทั้งไฟล์

12 ตัวเชื่อม — ชั้นที่สำคัญที่สุดและยังไม่มีเลย

สองฝั่งข้างบนต่อให้ทำครบก็ยังไม่ได้ ถ้าไฟล์สัญญาเดินทางข้ามฝั่งไม่ได้ นี่เป็นหัวข้อที่ต้องตัดสินใจก่อนอย่างอื่นทั้งหมด และมีรายละเอียดอยู่ในหัวข้อ 15

13 checklist รวม

ใช้ตารางนี้ตอบคำถาม “เรามีสัญญาแล้วหรือยัง” — ตอบว่ามีได้ก็ต่อเมื่อครบทุกข้อ

#	รายการ	เจ้าของ
1	ตกลงวิธีให้ไฟล์สัญญาเดินทางข้ามฝั่ง และทำจริงแล้ว	ทั้งทีม
2	zod เวอร์ชันเดียวกันทั้งสองฝั่ง	ทั้งทีม
3	ตกลงรูปร่างของ ctx.user แล้ว	backend
4	ตั้ง autoSchemaFile และมีไฟล์สัญญาถูกสร้างจริง	backend
5	เปิด CORS พร้อม credentials และทดสอบว่า cookie ไปถึง	backend
6	มี procedure 4 ระดับตาม role	backend
7	ทุก procedure มีทั้ง input และ output schema	backend
8	ไม่มี procedure ไหน return Prisma model ตรง ๆ	backend
9	frontend import type สัญญาได้และ tsc --noEmit ผ่าน	frontend
10	มีตารางสัญญาที่คนอ่านได้ อัปเดตตรงกับโค้ด	ทั้งทีม

ตารางที่ 6: checklist ความครบถ้วนของสัญญา — ข้อ 7 กับ 8 เป็นข้อที่ compiler จับให้ไม่ได้ ต้องใช้คนรีว

ภาค 4

ความเสี่ยงและข้อกำหนดในรีโปนี้

คำถามที่ภาคนี้ตอบ: อะไรในกองโค้ดที่เรามีอยู่ตอนนี้ ที่จะทำให้การทำสัญญา tRPC สะดุด และต้องแก้ก่อนหรือระวังตอนไหน

ทุกข้อในภาคนี้มาจากการเปิดไฟล์จริงในรีโปดู ไม่ใช่ความเสี่ยงตามตำรา แต่ละข้อบอก ว่าอาการเป็นอย่างไร ถ้าไม่จัดการจะเจออะไร และวิธีกัน

#	ความเสี่ยง	โอกาสเกิด	ความรุนแรง	ต้องจัดการเมื่อไร
1	zod คนละเวอร์ชัน (3 กับ 4)	แน่นอน	สูง	ก่อนเขียนบรรทัดแรก
2	คนละโปรเจกต์ npm คนละสาขา git	แน่นอน	สูงมาก	ก่อนเขียนบรรทัดแรก
3	schema เป็น PascalCase และไม่มี enum	แน่นอน	กลาง	ตอนออกแบบ output
4	DateTime กลายเป็นสตริง	แน่นอน	กลาง	ตอนออกแบบ output
5	อัปเดตรูปผ่าน tRPC ไม่ได้	แน่นอน	กลาง	ตอนทำ feature รับ-คืนของ
6	polling ถี่ปะทะ batching	สูง	กลาง	ตอนต่อหน้าที polling
7	ยังไม่ได้เปิด CORS + cookie	แน่นอน	สูง	วันแรกที่ต่อสองฝั่ง
8	ลืมนใส่ output แล้วได้ any	สูง	สูงมาก	ตลอดโครงการ

ตารางที่ 7: สรุปความเสี่ยงแปดข้อ — ข้อ 1, 2 และ 7 ต้องจบก่อนงานอื่นทั้งหมด ส่วนข้อ 8 เป็นความเสี่ยงที่ต้องเฝ้าตลอด

14 ความเสี่ยง 1 — zod คนละเวอร์ชัน

ตัวเลขจริงจากรีโป

backend-preview/backend/package.json ระบุ "zod": "^4.4.3"

frontend/package.json ระบุ "zod": "^3.23.8"

ไฟล์สัญญาที่ nestjs-trpc สร้างออกมาไม่ได้มีแค่ type ล้วน ๆ แต่มีโค้ด zod schema ประกอบอยู่ด้วย เมื่อ frontend เอาไป compile ด้วย zod คนละรุ่นใหญ่ จะได้ error ของ TypeScript ที่อ่านแล้วไม่รู้ว่าเป็นสาเหตุคือเรื่องเวอร์ชัน เพราะข้อความจะพูดถึงชนิดข้อมูลภายในของ zod เอง

การอัปเดต zod ฝั่ง frontend ไม่ใช่แค่เปลี่ยนตัวเลข

ต้องขยับของที่พึ่งพากันด้วย

- frontend/src/schemas/loan-request.schema.ts — schema ที่เขียนไว้แล้ว ต้องตรวจว่ายัง

ใช้ได้ (zod 4 เปลี่ยนรูปแบบข้อความ error และเปลี่ยน API บางตัว)

- @hookform/resolvers — ตัวเชื่อม react-hook-form กับ zod ต้องใช้รุ่นที่รองรับ zod 4
- ทุกที่ที่เรียก .errors หรืออ่านโครงสร้างผลลัพธ์ของ safeParse

ทำเรื่องนี้เป็นงานแยกให้จบและ tsc --noEmit ผ่านก่อน แล้วค่อยเริ่มงาน tRPC — ย่อรวมสองเรื่องนี้ไว้ใน PR เดียวกัน เพราะเวลาฟังจะแยกไม่ออกว่า ฟังเพราะอะไร

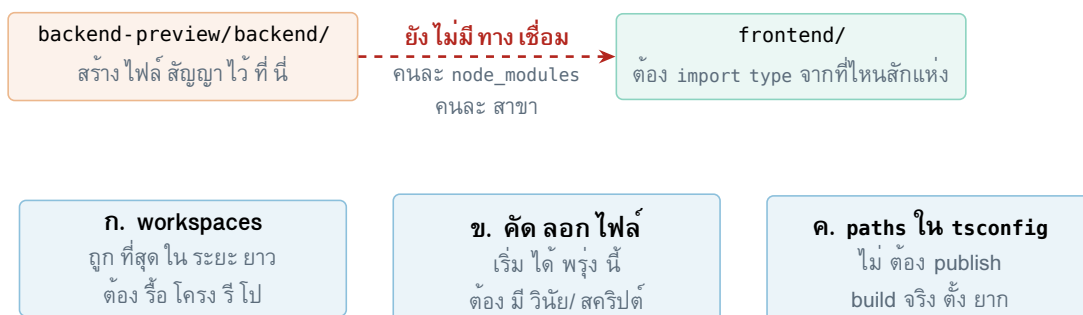
ทางเลือกสำรองคือ ให้ frontend ใช้ไฟล์สัญญาเป็น type อย่างเดียว โดยไม่ compile โค้ด zod ในนั้น ซึ่งลดแรงเสียดทานได้ แต่แลกมากับการที่ frontend เอา schema เดียวกันไปใช้ validate ฟอรั่มไม่ได้ ต้องเขียน schema ฟอรั่มซ้ำเอง — ซึ่งก็คือการเปิดช่องให้ความจริงสองชุดเลื่อนออกจากกันอีก

15 ความเสี่ยง 2 — คนละโปรเจกต์ npm คนละสาขา git

นี่คือความเสี่ยงที่บล็อกทุกอย่าง และเป็นวาระแรกของการประชุม

สภาพจริงของรีโป

มี package.json สามไฟล์ที่ไม่รู้จักกัน: frontend/, frontend-preview/frontend/ และ backend-preview/backend/ — ไม่มี pnpm-workspace.yaml ไม่มี workspaces ใน package.json ระดับบนสุด และงานสองฝั่งอยู่คนละสาขา (front-end-branch, back-end-branch)



ข้อเสนอ: เริ่มด้วย ข. พร้อมสคริปต์ npm run sync:contract แล้วค่อยขยับไป ก. ถ้ามีเวลาเหลือ

รูปที่ 7: สามทางเลือกของการขนส่งสัญญา — ต่างกันที่จ่ายตอนนี้หรือจ่ายทีหลัง ไม่มีทางไหนฟรี

ทางที่ห้ามเลือก: คัดลอกด้วยมือแล้วไม่มีอะไรตรวจ

ถ้าเลือกวิธีคัดลอกไฟล์ แต่ไม่มีสคริปต์และไม่มีกระบวนการตรวจตอน merge สัญญาฝั่ง frontend จะเก๋กว่าของจริงภายในไม่กี่วัน แล้วจะได้สิ่งที่แย่กว่าไม่มีสัญญาเลย คือ compiler ที่บอกว่าทุกอย่างเรียบร้อยทั้งที่ไม่จริง

อย่างน้อยที่สุดต้องมี: สคริปต์คัดลอก + tsc --noEmit ทั้งสองฝั่งเป็นเงื่อนไข ก่อน merge

16 ความเสี่ยง 3 — schema เป็น PascalCase และไม่มี enum เลย

นับจาก `schema.prisma` จริง

32 model และ 0 enum — สถานะทุกชนิดถูกทำเป็นตารางแยก: `ApproveStatus`, `ResourceStatus`, `ConditionType`, `ImageSubmissionType`, `NotificationType`, `PenaltyType`, `GroupType` และแม้แต่ `RoleInfo` ก็เป็นตาราง ไม่ใช่ enum

ขณะที่ `frontend/src/types/domain.ts` นิยาม `Role` ไว้เป็น `"borrower" | "staff" | "supervisor" | "admin"` และ `UnitStatus` เป็น `"available" | "reserved" | "borrowed" | "maintenance" | "lost"`

ความไม่ตรงกันนี้มีสองชั้น และต้องแก้คนละแบบ

ชั้น	อาการ	วิธีจัดการ
ชื่อฟิลด์	<code>UserFName</code> กับ <code>firstName</code>	แปลงใน output schema — เขียนครั้งเดียวต่อ entity แล้วเอาไป <code>.pick().extend()</code> ต่อ ไม่ต้องเขียนซ้ำทุก procedure
สถานะ	ฐานข้อมูลให้ <code>ApproveStatusKey</code> เป็นตัวเลข แต่ frontend อยากรู้คำ	output ต้องเป็น สตริงคงที่ เช่น <code>z.enum(['pending', 'approved', 'rejected'])</code> ห้าม ส่งเลข primary key ออกไปให้ frontend ดีความเอง

ตารางที่ 8: สองชั้นของความไม่ตรงกัน — ชั้นล่างอันตรายกว่า เพราะเลขคีย์เปลี่ยนได้เมื่อ seed ข้อมูลใหม่

ถ้าส่งเลข key ของสถานะออกไป frontend จะพังตอนย้ายเครื่อง

`ApproveStatusKey = 2` หมายถึง “อนุมัติแล้ว” บนฐานข้อมูลของเครื่องคนหนึ่ง แต่ถ้าอีกเครื่อง seed ข้อมูลคนละลำดับ เลข 2 อาจกลายเป็น “ปฏิเสธ”

โค้ดฝั่งหน้าเว็บที่เขียนว่า `if (status === 2)` จะยังคอมไพล์ผ่านทุกครั้ง และจะผิดเจียบ ๆ — แปลงเป็นสตริงที่ชั้น output เสมอ

17 ความเสี่ยง 4 — DateTime กลายเป็นสตริง

อธิบายกลไกไปแล้วในหัวข้อ 6 ที่ต้องเพิ่มคือขนาดของปัญหาในรีโปนี้: `schema.prisma` มีฟิลด์ `DateTime` กระจายอยู่ใน `Reservations`, `UsageLog`, `ExtensionRequest`, `Inspection`, `PenaltyInfo`, `AppealInfo`, `Images`, `Notification` และ `ConditionLog` — แทบทุกโดเมนหลักของระบบ

และเป็นระบบที่ “เวลา” คือแก่นของตรรกะธุรกิจ: กำหนดคืน การเลยกำหนด การหักเครดิต รายวัน อายุของบทลงโทษ วันหมดอายุคำขอ ถ้าตกลงรูปแบบเวลาไม่ตรงกัน ก็จะไปโผล่ที่ การคำนวณค่าปรับ ซึ่งเป็นที่ที่แย่

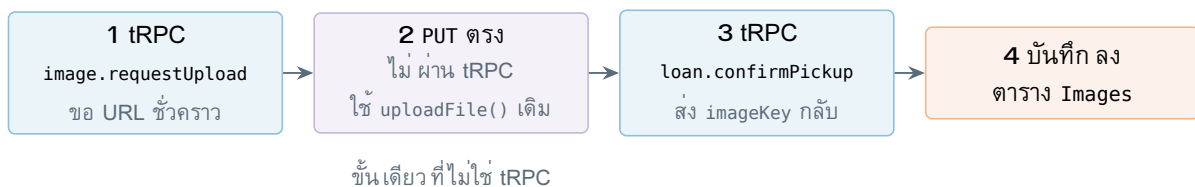
ที่สุดที่จะมีบั๊ก

ตกลงครั้งเดียว เขียนลงตารางสัญญา แล้วห้ามมีข้อยกเว้น

ไม่ว่าจะเลือก superjson หรือสตริง ISO สิ่งที่สำคัญกว่าคือ **ทั้งระบบต้องเหมือนกันหมด** — procedure ที่ส่ง Date จริง ปนกับ procedure ที่ส่งสตริง คือสถานการณ์ที่แย่มากที่สุดในสามแบบ

18 ความเสี่ยง 5 — อัปโหลดรูปผ่าน tRPC ไม่ได้

เอกสาร “รายการเรียกใช้งานจาก Backend” ระบุไว้สองรายการ: “ยืนยันรับของ + อัปโหลดรูปตอนรับ” และ “แจ้งคืน + อัปโหลดรูปตอนคืน” และ schema.prisma มีตาราง Images กับ ImageSubmissionType รองรับไว้แล้ว



รูปที่ 8: รูปแบบผสมสำหรับการอัปโหลด — tRPC คุมหัวและท้าย ส่วนตัวไฟล์ไปเส้นทางของมันเอง

จุดที่ต้องคิดต่อ: ไฟล์ที่อัปแล้วแต่ไม่มีใครอ้างถึง

ถ้าผู้ใช้อัปรูปสำเร็จ (ขั้น 2) แล้วปิดเบราว์เซอร์ก่อนขั้น 3 จะมีไฟล์ค้างอยู่โดยไม่มี แถวใน Images ซึ่ถึงต้องมี job เก็บกวาด หรือยอมรับว่ามีขยะสะสม — ตัดสินใจตอนทำ feature นี้ ไม่ต้องตัดสินใจตอนนี้

19 ความเสี่ยง 6 — polling ถึปะทะ batching

เอกสารรายการเรียกใช้งานกำหนด polling ไว้ 6 รายการ ที่ถี่ที่สุดคือ 10–15 วินาที (ของคงเหลือ และสล็อตเวลาว่างของห้อง) ซึ่งเป็นความถี่ที่ต้องคิดเรื่องภาระ ผัง backend ตั้งแต่ตอนออกแบบสัญญา ไม่ใช่ตอนพบว่าช้า

query	ความถี่	สิ่งที่ต้องเตรียม
ของคงเหลือ / วันที่พร้อมให้ยืม	10–15 วิ	output ต้องเล็กที่สุด — ส่งแค่จำนวนคงเหลือกับวันที่พร้อมยืม ไม่ต้องส่งรายละเอียดอุปกรณ์ซ้ำ
สล็อตเวลาว่างของห้อง (T3)	10–15 วิ	ต้องมี index บน Reservations ตามช่วงเวลา
สถานะคำขอของตัวเอง	30 วิ	กรองด้วย ctx.user เสมอ
notification ในแอป	30–60 วิ	ส่งเฉพาะจำนวนที่ยังไม่อ่านก็พอสำหรับ badge
คิวงานฝั่ง staff	30 วิ	แบ่งหน้าเสมอ อย่าส่งทั้งคิว
คิวรออนุมัติฝั่งอาจารย์	30–60 วิ	แบ่งหน้าเสมอ

ตารางที่ 9: รายการที่ polling จากเอกสารรายการเรียกใช้งาน — ความถี่เป็นข้อมูลที่ backend ต้องรู้ตั้งแต่ตอนออกแบบ output

ใส่คอลัมน์ “ความถี่ที่ตั้งใจ” ลงในตารางสัญญา

ความถี่ในการเรียกไม่ใช่เรื่องของ frontend ฝ่ายเดียว มันคือข้อกำหนดที่ backend ต้องออกแบบ query และ index รองรับ — จึงเป็นส่วนหนึ่งของ สัญญา และต้องอยู่ในตารางเดียวกับ input/output

20 ความเสี่ยง 7 — ยังไม่ได้เปิด CORS และยังไม่มีเส้นทางของ cookie

ตรวจสอบกับรีโปแล้ว

backend- preview/ backend/ src/ main.ts มีแค่ NestFactory.create กับ app.listen — ไม่มี app.enableCors() ไม่มีการอ่าน cookie ขณะที่ frontend/ src/ lib/ api- client.ts ตั้ง credentials: "include" ไว้แล้วทุก request แปลว่าฝั่งหน้าเว็บตัดสินใจเรื่องนี้ไปแล้ว

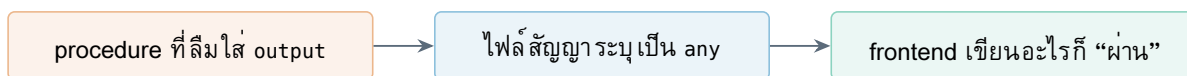
วันแรกที่เอาสองฝั่งมาต่อกัน อาการที่จะเจอคือ request ถูกบล็อกโดยเบราว์เซอร์ พร้อมข้อความ CORS ซึ่งมักถูกแก้แบบผิด ๆ ด้วยการตั้ง origin: '*'

origin: '*' ใช้กับ credentials: true ไม่ได้ และไม่ใช้แค่เพราะเบราว์เซอร์ห้าม

เบราว์เซอร์ปฏิเสธการส่ง cookie ไปยัง origin แบบ wildcard อยู่แล้ว แต่ถึงแม้จะบังคับได้ ก็ไม่ควรทำ เพราะระบบนี้ใช้ cookie เป็นตัวยืนยันตัวตน การเปิด origin กว้างคือการเปิดช่องให้เว็บอื่นยิง request ในนามผู้ใช้ที่ ล็อกอินอยู่ (CSRF) ต้องระบุ origin ให้ชัดเจน และต้องมีรายการแยกสำหรับตอนพัฒนา (http://localhost:5173 ของ Vite) กับตอนใช้งานจริง

21 ความเสี่ยง 8 — ลืมใส่ output แล้วได้ any

นี่คือความเสี่ยงที่อันตรายที่สุด เพราะมันทำลายเหตุผลทั้งหมดของการใช้ tRPC โดยไม่มีอะไรเสี่ยงเตือน



tsc ไม่ ฟ้อง eslint ไม่ ฟ้อง เทสต์ ก็ ผ่าน เพราะ any เข้า กัน ได้ กับ ทุก อย่าง
จะรู้ตัวก็ตอนผู้ใช้กดแล้วหน้าขาว — ซึ่งเป็นสภาพเดียวกับตอนที่ยังไม่มีสัญญาเลย แต่แย่กว่าตรงที่ทีมเชื่อไปแล้วว่ามี

รูปที่ 9: ทางที่สัญญาพังเงียบ — any หนึ่งตัวทำให้ทั้งเส้นทางไม่มีการตรวจสอบเหลืออยู่

วิธีกันสามชั้น เรียงจากถูกที่สุดไปแพงที่สุด

1. กฎในการรีวิว PR — “ทุก @Query/@Mutation ต้องมีทั้ง input และ output” เขียนไว้ในเอกสารสัญญา และใน template ของ PR
2. เปิด `noImplicitAny` และเปิดกฎ ESLint ที่ห้าม any ชัดแจ้ง ผัง frontend — จับได้บางกรณี ไม่ใช่ทุกกรณี
3. `tsc --noEmit` ทั้งสองฝั่งเป็นเงื่อนไขก่อน merge — จับกรณีสัญญาเก่าไม่ตรงกับโค้ด ซึ่งเป็นอาการที่พบบ่อยกว่า

สัญญาที่ไม่มีใครบังคับ จะเน่าภายในสองสัปดาห์

ข้อนี้ไม่ใช่เรื่องเทคนิค แต่เป็นเรื่องกระบวนการ และเป็นข้อที่ทีมนักศึกษาพลาดบ่อยที่สุด เพราะช่วงใกล้เดดไลน์ทุกคนจะยอมข้ามการตรวจ — ต้องทำให้การตรวจเป็นอัตโนมัติตั้งแต่ ตอนที่ยังไม่มี

ภาค 5

ต้องทำอะไรบ้าง ตามลำดับ

คำถามที่ภาคนี้ตอบ: ฟรังก์ชันนี้เข้าเปิดคอมมา ทำอะไรก่อน และจะรู้ได้อย่างไรว่าทำเสร็จแล้ว

22 ลำดับงานที่เสนอ



รูปที่ 10: ลำดับงาน — ขั้นที่ 1 เป็นคอขวดโดยตั้งใจ หลังจากนั้นสองฝั่งทำงานขนานกันได้เต็มที่

22.1 ทำไมต้องทำเส้นเดียวให้ครบก่อน

สัญญาขออนุญาตของทีมมักบอกให้ “เขียน router ให้ครบทุกโดเมนก่อน แล้วค่อยต่อ frontend” ซึ่งเป็นลำดับที่แพงที่สุด เพราะปัญหาโครงสร้าง (cookie ไม่ไป ถึง, สัญญาข้ามฝั่งไม่ได้, zod ขนกัน) จะไปโผล่ตอนที่เขียนไปแล้ว 30 procedure และต้องแก้ทั้ง 30 ตัว

การทำเส้นเดียวให้วิ่งครบ (vertical slice) เจอปัญหาเดียวกันทั้งหมดตั้งแต่ procedure แรก ตอนที่แก้แล้วกระแทกแค่ตัวเดียว

เกณฑ์ว่าขั้นที่ 1 เสร็จ

เปิดหน้าเว็บที่ยังไม่ล็อกอิน → ถูกดึงไปหน้า login → ล็อกอิน → หน้าแสดงชื่อผู้ใช้จริงจากฐานข้อมูล และเมื่อลองเปลี่ยนชื่อฟิลด์ใน output schema ฝั่ง backend แล้วรัน `tsc --noEmit` ฝั่ง frontend ต้องขึ้น error

ประโยคหลังคือการทดสอบว่าสัญญาทำงานจริง ไม่ใช่แค่ข้อมูลวิ่งถึง

22.2 stub ที่คืนค่า mock — ประโยชน์ที่แท้จริงของการทำสัญญาก่อน

หลังตกลง output schema ของ procedure หนึ่งแล้ว backend สามารถ เขียน procedure ที่ return ข้อมูลปลอมที่ตรงตาม schema ได้ทันที โดยยังไม่ต้องแตะฐานข้อมูล

```
@Query({ input: listLoansInput, output: listLoansOutput })
list(@Input() input: ListLoansInput) {
  // TODO(backend): replace with loanService.listForUser once ready
  return {
    items: [
      { id: 1, itemName: 'Canon EOS R5', dueAt: '2569-08-20T09:00:00Z',
        status: 'borrowed' as const },
    ],
    total: 1, page: 1, pageSize: 20,
  };
}
```

frontend ต่อบรรทัดได้เลยโดยไม่ต้องรอฐานข้อมูล และเมื่อ backend เปลี่ยน มาใช้ของจริง frontend ไม่ต้องแก้ไขอะไรเลยถ้า output schema ไม่เปลี่ยน — นี่คือเหตุผลทั้งหมดที่เราลงทุนทำสัญญาก่อน

23 กติกาที่ต้องบังคับ

#	กติกา	บังคับด้วยอะไร
1	ทุก procedure มีทั้ง input และ output schema	คนรีวิว PR
2	ห้าม return Prisma model ตรง ๆ	คนรีวิว PR
3	สถานะส่งออกเป็นสตริงคงที่ ไม่ใช่เลข key	คนรีวิว PR
4	tsc --noEmit ผ่านทั้งสองฝั่ง	อัตโนมัติก่อน merge
5	แก้สัญญาแล้วต้องรันสคริปต์ sync	อัตโนมัติ + คนรีวิว
6	เปลี่ยนสัญญาแบบ breaking ต้องแจ้งอีกฝั่งก่อน	ข้อตกลงของทีม

ตารางที่ 10: หกกติกา — สีข้อแรกเป็นเรื่องความถูกต้อง สองข้อหลังเป็นเรื่องไม่ทำให้เพื่อนร่วมทีมงานพัง

24 Definition of Done ของ procedure หนึ่งตัว

ใช้เป็น checklist ตอนปิด issue แต่ละอัน

#	เกณฑ์
1	มีแถวในตารางสัญญา ระบุ input, output, role ที่เรียกได้, error ที่เป็นไปได้
2	มี input schema ที่ครอบคลุมค่าขอบ (ค่าว่าง ค่าติดลบ สตริงยาวเกิน)
3	มี output schema ที่ไม่มีฟิลด์ภายในของฐานข้อมูลหลุดออกไป
4	ผูก middleware ตาม role ที่ระบุในตารางสัญญาแล้ว
5	error ทางธุรกิจถูกโยนด้วยรหัสที่อยู่ในรายการที่ตกลงกันไว้ ไม่ใช่ข้อความไทยดิบ
6	frontend เรียกได้จริง และ tsc --noEmit ผ่านทั้งสองฝั่ง
7	ถ้าเป็น mutation — ระบุแล้วว่าต้อง invalidate query ตัวไหนบ้าง
8	ถ้าเป็น query ที่ polling — ระบุความถี่ และตรวจแล้วว่า output ไม่ใหญ่เกินจำเป็น

ตารางที่ 11: Definition of Done — ข้อ 7 กับ 8 เป็นข้อที่ทีมมักลืม แล้วไปพบว่าหน้าจอไม่อัปเดตหลังกดปุ่ม

สรุปทั้งเล่มในสามประโยค

1. สัญญาของ tRPC คือ type ของ TypeScript — มีผลตอน compile เท่านั้น ตอน runtime ต้องพึ่ง zod
2. สัญญาจะมีประโยชน์ก็ต่อเมื่อ **เดินทางข้ามฝั่งได้จริง** และ **มีคนบังคับให้ตรงกัน** — สองเรื่องนี้เป็นเรื่องกระบวนการ ไม่ใช่เรื่องโค้ด
3. สิ่งที่ต้องตกลงก่อนเขียนบรรทัดแรกมีสี่เรื่อง และอยู่ในเอกสารวาระประชุม ฉบับคู่กัน

ภาคผนวก

ก อภิธานศัพท์ ไทย-อังกฤษ

ผู้อ่านจะเจอศัพท์อังกฤษเหล่านี้ในเอกสารของไลบรารี ในข้อความ error และในโค้ด ตารางนี้จึงจำเป็น — การแปลเป็นไทยอย่างเดียวจะตัดผู้อ่านออกจากแหล่งข้อมูลจริง

ไทย	อังกฤษ	ความหมายย่อ
สัญญา	contract	ข้อตกลงเรื่องรูปร่างข้อมูลระหว่างสองฝั่ง ใน tRPC คือ type ของ AppRouter
ตัวจัดเส้นทาง	router	กลุ่มของ procedure ที่จัดไว้ด้วยกันตามโดเมน เช่น loan, item
หน่วยเรียกใช้	procedure	หนึ่งการเรียกที่ frontend ใช้ได้ แบ่งเป็น query กับ mutation
การอ่าน	query	procedure ที่อ่านอย่างเดียว ไม่เปลี่ยนข้อมูล จึง cache และเรียกซ้ำได้ปลอดภัย
การเปลี่ยนแปลง	mutation	procedure ที่เขียนข้อมูล เรียกซ้ำแล้วผลไม่เหมือนเดิม
บริบทของคำขอ	context	object ที่สร้างใหม่ทุก request เก็บสิ่งที่ทุก procedure ต้องใช้ เช่น ctx.user
ชั้นคั่นกลาง	middleware	โค้ดที่ทำงานก่อนเข้า procedure ใช้บังคับสิทธิ์ และการล็อกอิน
ชั้นส่งออก	link	ชั้นที่ตัดสินใจว่าจะส่ง call ออกไปอย่างไร เช่น httpBatchLink
การรวมคำขอ	batching	รวมหลาย query ที่เกิดพร้อมกันให้เป็น HTTP request เดียว
การถามซ้ำเป็นรอบ	polling	เรียก query เดิมซ้ำตามเวลาที่กำหนด เพื่อให้ข้อมูลใหม่
การทำให้ข้อมูลหมดอายุ	invalidate	บอก TanStack Query ว่าข้อมูลชุดนี้เก่าแล้ว ให้ไปดึงใหม่
การอนุมานชนิดข้อมูล	type inference	การที่ TypeScript คิดชนิดข้อมูลออกเองโดยไม่ต้องเขียนกำกับ
ชนิดข้อมูลเลื่อน	type drift	อาการที่ type ฝั่งหนึ่งกับอีกฝั่งค่อย ๆ ไม่ตรงกันโดยไม่มีใครรู้

ไทย	อังกฤษ	ความหมายย่อ
การตรวจสอบความถูกต้อง	validation	ตรวจสอบว่าข้อมูลที่ได้รับมาถูกรูปแบบจริงตอน runtime — งานของ zod
เส้นเดียวครบวงจร	vertical slice	ทำหนึ่ง feature ให้วิ่งครบตั้งแต่หน้าจอลงถึงฐานข้อมูล ก่อนขยายจำนวน
ข้อมูลจำลอง	mock / stub	ค่าปลอมที่ตรงตาม schema ใช้ให้ frontend ทำงานต่อได้ระหว่างรอของจริง
การแบ่งหน้า	pagination	ส่งข้อมูลที่ละหน้าแทนที่จะส่งทั้งหมด
ตัวแปลงข้อมูล	transformer	ตัวที่แปลงชนิดข้อมูลที่ JSON ส่งไม่ได้ เช่น superjson จัดการ Date

ข ร่างตารางสัญญาที่ต้องทำให้เสร็จ

ตารางนี้คือ **รูปแบบ** ที่เสนอ ไม่ใช่เนื้อหาที่เสร็จแล้ว — เนื้อหาจริงต้อง แปลงมาจากเอกสาร “รายการเรียกใช้งานจาก Backend” ทั้ง 18 รายการ บวกกับหน้าจอ ใน frontend/src/features/ ที่มีอยู่ 6 กลุ่ม (admin, staff, supervisor, borrower, reports, auth)

procedure	ชนิด	input	output	role	หมายเหตุ
auth.me	query	—	userOutput	ล็อกอินแล้ว	ทุกหน้าใช้
item.list	query	q, page, filters	items[], total	ล็อกอินแล้ว	แบ่งหน้า
item.availability	query	itemId	available, nextAt	ล็อกอินแล้ว	poll 10–15 วิ
loan.create	mutation	createLoanInput	loanOutput	ผู้ยืม	ITEM_UNAVAILABLE, LOAN_PERIOD_EXCEEDS_LIMIT
loan.myActive	query	—	loans[]	ผู้ยืม	invalidate หลังทุก action
approval.queue	query	page	requests[], total	อาจารย์	poll 30–60 วิ
approval.decide	mutation	id, decision, reason	ok	อาจารย์	ALREADY_DECIDED

ตารางที่ 13: ตัวอย่างรูปแบบของตารางสัญญา — คอลัมน์ role กับหมายเหตุคือสิ่งที่ทำให้ตารางนี้ต่างจากรายการ endpoint ธรรมดา

ค คำสั่งที่ต้องรันจริง

1. `cd frontend && npm i zod@^4` — แก้ความเสี่ยงข้อ 1 แล้วรัน `npm run typecheck` ให้ผ่านก่อนทำอย่างอื่น
2. `cd frontend && npm i @trpc/client` (พร้อมตัวเชื่อม TanStack Query รุ่นที่เข้าชุดกับ `@trpc/server 11`)
3. ตั้ง `autoSchemaFile` ใน `backend-preview/backend/src/app.module.ts` แล้ว `npm run start:dev` — ตรวจสอบว่าไฟล์สัญญาถูกสร้างขึ้นจริง
4. `cd backend-preview/backend && npx prisma generate` — ถ้ายังไม่เคยรัน `src/generated/prisma` จะยังไม่มี client
5. `npm run sync:contract` — สคริปต์คัดลอกไฟล์สัญญา (ยังไม่มี ต้องเขียน หลังตกลงวิธีขนส่งในการประชุม)
6. สร้างเอกสารชุดนี้ใหม่: `cd docs && latexmk trpc-guide.tex`

ง ที่มาของข้อเท็จจริงทุกข้อในเอกสารนี้

เอกสารนี้ไม่มีผลการทดลอง มีแต่ข้อเท็จจริงที่อ่านจากไฟล์ในรีโป ตารางนี้คือรายการ ไฟล์ที่ถูกอ่าน — ถ้าไฟล์ไหนเปลี่ยน เนื้อความในหัวข้อที่ระบุต้องถูกตรวจซ้ำ

ไฟล์	เอกสารนี้ใช้มันยืนยันอะไร
backend-preview/backend/package.json	เวอร์ชัน nestjs-trpc 2.13, @trpc/server 11.18, zod 4.4, @prisma/client 7.9 — ความเสี่ยงข้อ 1 และภาค 3
.../src/app.module.ts	TRPCModule.forRoot() เรียกแบบไม่ส่ง option — ภาค 3
.../src/main.ts	ยังไม่มี enableCors — ความเสี่ยงข้อ 7
.../prisma/schema.prisma	32 model, 0 enum, ชื่อฟิลด์ PascalCase, HashedPassword อยู่ในตารางเดียวกับข้อมูลผู้ใช้, ฟิลด์ DateTime กระจาย 9 ตาราง — ความเสี่ยงข้อ 3 และ 4
frontend/package.json	zod 3.23, @tanstack/react-query 5.59, ไม่มี @trpc/client — ความเสี่ยงข้อ 1 และภาค 3
frontend/src/lib/api-client.ts	เลือกใช้ httpOnly cookie และ credentials: "include" ไปแล้ว, รูปแบบ ApiError — ความเสี่ยงข้อ 7
frontend/src/types/domain.ts	type เขียนมือแบบ camelCase และ union ของสถานะ — ความเสี่ยงข้อ 3
รายการเรียกใช้งานจาก Backend.pdf	รายการ endpoint 3 กลุ่ม, ความถี่ polling 6 รายการ, งาน cron 7 งาน — ความเสี่ยงข้อ 5 และ 6

ตารางที่ 14: ที่มาของข้อเท็จจริง — ตารางนี้คือสิ่งที่ทำให้เอกสารนี้ตรวจสอบย้อนกลับได้ ไม่ใช่แค่ความเห็น

เอกสารภายในโครงการที่ควรอ่านคู่กัน

- docs/trpc-meeting.tex — วาระประชุมสำหรับตกลงสิ่งที่เอกสารนี้ชี้ว่าต้องตกลง
- รายการเรียกใช้งานจาก Backend.pdf — ร่างสัญญาฉบับแรกของทีม ยังใช้เป็นตัวตั้งได้
- docs/PROGRESS.md — สถานะสะสมของงานเอกสาร

รูปทุกรูปในเอกสารนี้วาดด้วย TikZ ในไฟล์ docs/trpc-guide.tex เอง ไม่มีรูปที่มาจากกรรณโศก และไม่มีรูปที่เป็นผลการทดลอง

ข้อเท็จจริงทุกข้อตรวจสอบย้อนกลับได้ตามตารางในภาคผนวก ง สถานะที่อ้างถึงเป็นสถานะ ณ วันที่ระบุบนหน้าปก